New chat
Search
Customize

Products
Code
Design

Starred
Launching Bangladesh's first AI-...
ATOS - Main Chat

Recents
Humanizing technical texts to a...
Portfolio activities analysis
MentorXchange test case valida...
Preparing Nexus presentation f...
AI solutions for Hong Kong Polic...
Building OCP's proprietary data...
Converting PDF to LaTeX format
AI training modules for executiv...
Learning the platform thoroughly
Canadian market expansion and...

i choose your second option "CVE-2026-42208". So tell me the full steps to do this from the absolute beginning because you have to assure that i haven't done this before and don't know how this works. And i desperate need to finish this asap. Remember, I need to do the fix myself and show that the fix works as well for this activity, so give me all the steps from the absolute start. I'll be using Kali Linux inside a VM so everything you suggest must re aligned with this

Architected simplified PoC approach over complex server setup >

Good. Now I have everything I need to give you a complete, reliable guide. Here's the important thing I want to flag upfront: the full LiteLLM proxy requires PostgreSQL + Prisma setup which is complex. The smartest approach for this assignment is to reproduce the **exact vulnerable code pattern** from CVE-2026-42208 in a standalone Python script (using SQLite, which needs zero installation). This is completely legitimate academically because the vulnerability class (string interpolation in SQL), the fix (parameterized queries), and the AI comparison are all real and accurate. Let's go.

Complete Step-by-Step Guide: CVE-2026-42208 on Kali Linux

PHASE 1: Set Up Your Environment

Step 1 - Open a terminal on your Kali VM. Click the terminal icon or press `Ctrl+Alt+T`.

Step 2 - Update Kali first.

```bash
sudo apt update && sudo apt upgrade -y
```

This takes a few minutes. Let it finish.

Step 3 - Confirm Python 3 is installed (it should already be on Kali).

```bash
python3 --version
pip3 --version
```

You should see something like `Python 3.12.x`. If pip is missing:

```bash
sudo apt install python3-pip -y
```

Step 4 - Create a dedicated project folder.

```bash
mkdir ~/cve-2026-42208
cd ~/cve-2026-42208
```

Step 5 - Create a Python virtual environment. This keeps your project isolated from the rest of the system.

```bash
python3 -m venv venv
source venv/bin/activate
```

Your terminal prompt will now show `(venv)` at the start. This means the virtual environment is active. All pip installs from here stay inside this folder.

PHASE 2: Install the Vulnerable Version of LiteLLM

Step 6 - Install the specific vulnerable version.

```bash
pip install litellm==1.83.6
```

This downloads LiteLLM version 1.83.6 -- the last vulnerable version before the fix. It will take 1-2 minutes.

Step 7 - Confirm it installed.

```bash
pip show litellm
```

You should see `Version: 1.83.6`.

Step 8 - Find the vulnerable source code inside the installed package.

```bash
find venv/lib -name "utils.py" -path "*/proxy/*"
```

This will output a path like: `venv/lib/python3.12/site-packages/litellm/proxy/utils.py`

Step 9 - Search for the SQL injection in the source code.

```bash
grep -n "bearer_token\|token\|f\"SELECT\|f'SELECT\|execute.*token" venv/lib/python3.
```

You are looking for lines that construct SQL using an f-string (like `f"SELECT ... '{token}'"`) instead of using `?` or `$1` placeholders. Screenshot whatever output this gives you -- this is your proof that the vulnerable code exists in v1.83.6.

Take a screenshot of this terminal output. Label it **"Screenshot 1: Vulnerable code in LiteLLM 1.83.6"**.

PHASE 3: Create a Proof of Concept (PoC) Script

This script accurately reproduces the exact SQL injection pattern from CVE-2026-42208 using Python's built-in SQLite. No server needed.

Step 10 - Create the PoC file.

```bash
nano poc_cve_2026_42208.py
```

Paste in the following exactly:

```python
import sqlite3

print("=" * 60)
print("CVE-2026-42208 - LiteLLM SQL Injection PoC")
print("Affected versions: 1.81.16 through 1.83.6")
print("=" * 60)

# ------------------------------------------------------------
# Simulate the LiteLLM database (LiteLLM_VerificationToken)
# ------------------------------------------------------------
conn = sqlite3.connect(':memory:')
conn.execute("""
    CREATE TABLE LiteLLM_VerificationToken (
        token TEXT,
        user_id TEXT,
        permissions TEXT
    )
""")
conn.execute("INSERT INTO LiteLLM_VerificationToken VALUES ('sk-real-admin-key', 'ad
conn.execute("INSERT INTO LiteLLM_VerificationToken VALUES ('sk-user-key-456', 'regu
conn.commit()

# ------------------------------------------------------------
# VULNERABLE FUNCTION
# This mirrors the actual pattern in LiteLLM proxy/utils.py
# before the fix in v1.83.7
# The bearer_token value is dropped DIRECTLY into the SQL string
# ------------------------------------------------------------
def check_key_VULNERABLE(bearer_token):
    cursor = conn.cursor()
    # THIS IS THE VULNERABLE LINE - f-string injects user input into SQL
    query = f"SELECT * FROM LiteLLM_VerificationToken WHERE token = '{bearer_token}'
    print(f"\n[DEBUG] SQL query being executed:\n  {query}")
    cursor.execute(query)
    return cursor.fetchone()

# ------------------------------------------------------------
# FIXED FUNCTION
# This mirrors the fix applied in v1.83.7
# The bearer_token is passed as a parameter - never inserted into query text
# ------------------------------------------------------------
def check_key_FIXED(bearer_token):
    cursor = conn.cursor()
    # THIS IS THE FIXED LINE - parameterized query, input is separate from SQL
    query = "SELECT * FROM LiteLLM_VerificationToken WHERE token = ?"
    print(f"\n[DEBUG] SQL query being executed:\n  {query}")
    print(f"[DEBUG] Parameter passed separately: {bearer_token}")
    cursor.execute(query, (bearer_token,))
    return cursor.fetchone()

# ------------------------------------------------------------
# TEST 1: Normal request with a wrong key (baseline)
# ------------------------------------------------------------
print("\n--- TEST 1: Normal wrong key (should fail) ---")
normal_wrong_key = "sk-fake-key-doesnt-exist"

result = check_key_VULNERABLE(normal_wrong_key)
if result:
    print(f"[VULNERABLE] Authenticated as: {result}")
else:
    print("[VULNERABLE] Result: No match found - correctly rejected")

# ------------------------------------------------------------
# TEST 2: SQL Injection attack on VULNERABLE version
# This is what an attacker sends in the Authorization: Bearer header
# ------------------------------------------------------------
print("\n--- TEST 2: SQL INJECTION ATTACK (CVE-2026-42208) ---")
sql_injection_payload = "' OR '1'='1"

print(f"Attacker sends Authorization: Bearer {sql_injection_payload}")
result = check_key_VULNERABLE(sql_injection_payload)
if result:
    print(f"[VULNERABLE] ATTACK SUCCEEDED - authenticated as: {result[1]}")
    print(f"[VULNERABLE] BYPASSED authentication with permissions: {result[2]}")
else:
    print("[VULNERABLE] Attack failed - not vulnerable")

# ------------------------------------------------------------
# TEST 3: Same SQL injection on FIXED version
# ------------------------------------------------------------
print("\n--- TEST 3: SAME ATTACK ON FIXED VERSION (v1.83.7+) ---")
print(f"Attacker sends Authorization: Bearer {sql_injection_payload}")
result = check_key_FIXED(sql_injection_payload)
if result:
    print(f"[FIXED] ATTACK SUCCEEDED - authenticated as: {result[1]}")
else:
    print("[FIXED] Attack BLOCKED - injection payload treated as a literal string, n

print("\n" + "=" * 60)
print("SUMMARY")
print("=" * 60)
print("Vulnerable (v1.83.6): SQL injection bypasses authentication")
print("Fixed (v1.83.7+):     Parameterized query blocks injection")
```

Save and exit: press `Ctrl+X`, then `Y`, then `Enter`.

Step 11 - Run the PoC.

```bash
python3 poc_cve_2026_42208.py
```

You will see output like this:

```
============================================================
CVE-2026-42208 - LiteLLM SQL Injection PoC
Affected versions: 1.81.16 through 1.83.6
============================================================

--- TEST 1: Normal wrong key (should fail) ---
[DEBUG] SQL query being executed:
  SELECT * FROM LiteLLM_VerificationToken WHERE token = 'sk-fake-key-doesnt-exist'
[VULNERABLE] Result: No match found - correctly rejected

--- TEST 2: SQL INJECTION ATTACK (CVE-2026-42208) ---
Attacker sends Authorization: Bearer ' OR '1'='1
[DEBUG] SQL query being executed:
  SELECT * FROM LiteLLM_VerificationToken WHERE token = '' OR '1'='1'
[VULNERABLE] ATTACK SUCCEEDED - authenticated as: admin_user
[VULNERABLE] BYPASSED authentication with permissions: all

--- TEST 3: SAME ATTACK ON FIXED VERSION (v1.83.7+) ---
Attacker sends Authorization: Bearer ' OR '1'='1
[DEBUG] SQL query being executed:
  SELECT * FROM LiteLLM_VerificationToken WHERE token = ?
[DEBUG] Parameter passed separately: ' OR '1'='1
[FIXED] Attack BLOCKED - injection payload treated as a literal string, no match
found
```

Take a screenshot of this entire output. Label it "Screenshot 2: PoC output - vulnerability and fix demonstrated".

PHASE 4: Ask Three AI Systems to Fix It

This is the main deliverable of B29. You will ask ChatGPT, Gemini, and Claude the exact same prompt and compare their answers.

Step 12 - Your exact prompt to use in all three AI systems:

Open ChatGPT (chatgpt.com), Gemini (gemini.google.com), and Claude (claude.ai) in separate browser tabs.

Paste this prompt into each one:

> I am researching CVE-2026-42208, a critical SQL injection vulnerability in LiteLLM Proxy versions 1.81.16 through 1.83.6. The vulnerability exists because user-supplied input from the Authorization: Bearer header is directly interpolated into an SQL query string instead of using parameterized queries.
> Here is the vulnerable code pattern:
>
> ```python
> def check_api_key(bearer_token):
>     cursor = conn.cursor()
>     query = f"SELECT * FROM LiteLLM_VerificationToken WHERE token = '{bearer_t
>     cursor.execute(query)
>     return cursor.fetchone()
> ```
>
> An attacker can send `Authorization: Bearer ' OR '1'='1` and bypass authentication entirely.
> Please:
>
> 1. Explain why this code is vulnerable
> 2. Show me the fixed version of this function
> 3. Explain why the fix prevents the SQL injection
> 4. Are there any additional security measures you would recommend?

Step 13 - Screenshot each AI's response. Label them:

- "Screenshot 3a: ChatGPT fix for CVE-2026-42208"
- "Screenshot 3b: Gemini fix for CVE-2026-42208"
- "Screenshot 3c: Claude fix for CVE-2026-42208"

Step 14 - Compare the three responses. Create a notes file:

```bash
nano ai_comparison.txt
```

Fill it in like this as you compare:

```
CVE-2026-42208 - AI Fix Comparison

CHATGPT:
- Fix approach: [what method they suggested]
- Code provided: [yes/no]
- Additional recommendations: [list them]

GEMINI:
- Fix approach: [what method they suggested]
- Code provided: [yes/no]
- Additional recommendations: [list them]

CLAUDE:
- Fix approach: [what method they suggested]
- Code provided: [yes/no]
- Additional recommendations: [list them]

CONSISTENCY ANALYSIS:
- Did all three agree on the fix? [yes/no]
- Where did they differ? [note any differences]
```

Save: `Ctrl+X`, `Y`, `Enter`.

PHASE 5: Apply the Fix -- Two Ways

Way 1: Apply the fix in code (preferred for proof)

Step 15 - Create the fixed version of the script.

```bash
nano fixed_cve_2026_42208.py
```

Paste in the following:

```python
import sqlite3

print("=" * 60)
print("CVE-2026-42208 - FIXED VERSION (Parameterized Queries)")
print("Fix applied as in LiteLLM v1.83.7+")
print("=" * 60)

conn = sqlite3.connect(':memory:')
conn.execute("""
    CREATE TABLE LiteLLM_VerificationToken (
        token TEXT,
        user_id TEXT,
        permissions TEXT
    )
""")
conn.execute("INSERT INTO LiteLLM_VerificationToken VALUES ('sk-real-admin-key', 'ad
conn.commit()

# FIXED: Parameterized query -- input is NEVER part of the SQL text
def check_api_key_fixed(bearer_token: str):
    cursor = conn.cursor()
    query = "SELECT * FROM LiteLLM_VerificationToken WHERE token = ?"
    cursor.execute(query, (bearer_token,))
    return cursor.fetchone()

# Test 1: valid key
print("\nTest 1: Valid key")
result = check_api_key_fixed("sk-real-admin-key")
print(f"Result: {'Authenticated as ' + result[1] if result else 'Rejected'}")

# Test 2: invalid key
print("\nTest 2: Invalid key")
result = check_api_key_fixed("sk-fake-key")
print(f"Result: {'Authenticated as ' + result[1] if result else 'Correctly Rejected'

# Test 3: SQL injection attempt
print("\nTest 3: SQL injection payload (' OR '1'='1)")
result = check_api_key_fixed("' OR '1'='1")
print(f"Result: {'Authenticated as ' + result[1] if result else 'ATTACK BLOCKED - Co

# Test 4: another injection attempt
print("\nTest 4: SQL injection payload (' OR 1=1--)")
result = check_api_key_fixed("' OR 1=1--")
print(f"Result: {'Authenticated as ' + result[1] if result else 'ATTACK BLOCKED - Co

print("\n" + "=" * 60)
print("All injection attempts blocked by parameterized queries.")
print("=" * 60)
```

Save and exit: `Ctrl+X`, `Y`, `Enter`.

Step 16 - Run the fixed version.

```bash
python3 fixed_cve_2026_42208.py
```

Expected output:

```
============================================================
CVE-2026-42208 - FIXED VERSION (Parameterized Queries)
Fix applied as in LiteLLM v1.83.7+
============================================================

Test 1: Valid key
Result: Authenticated as admin_user

Test 2: Invalid key
Result: Correctly Rejected

Test 3: SQL injection payload (' OR '1'='1)
Result: ATTACK BLOCKED - Correctly Rejected


Test 4: SQL injection payload (' OR 1=1--)
Result: ATTACK BLOCKED - Correctly Rejected

============================================================
All injection attempts blocked by parameterized queries.
============================================================
```

Take a screenshot. Label it "Screenshot 4: Fixed version - all injection attempts blocked".

Way 2: Upgrade the actual LiteLLM package (do this too)

Step 17 - Upgrade to the patched version.

```bash
pip install litellm==1.83.10
```

Step 18 - Verify the version changed.

```bash
pip show litellm
```

You will see `Version: 1.83.10`.

Step 19 - Confirm the fix is in the source code.

```bash
grep -n "bearer_token\|token\|execute.*token" venv/lib/python3.*/site-packages/litel
```

You should now see parameterized queries (`?` or `$1` placeholders) instead of f-strings. This confirms the fix is present in the package.

Take a screenshot. Label it "Screenshot 5: pip show litellm showing version 1.83.10 (patched)".

PHASE 6: Final Proof Side-by-Side

Step 20 - Run both scripts one after the other so you can see the contrast.

In the same terminal session:

```bash
echo "=== VULNERABLE VERSION OUTPUT ===" && python3 poc_cve_2026_42208.py
```

Then:

```bash
echo "=== FIXED VERSION OUTPUT ===" && python3 fixed_cve_2026_42208.py
```

Take a final screenshot showing both outputs. Label it "Screenshot 6: Before vs After - proof of fix".

PHASE 7: Summary of What You Have for Submission

Here is everything you now have for B29:

Item	What it proves
Screenshot 1	Vulnerable code exists in v1.83.6 source
Screenshot 2	PoC showing attack succeeds on vulnerable version
Screenshot 3a/3b/3c	Three AI systems each generating a fix
ai_comparison.txt	Your written comparison of the three AI responses
Screenshot 4	Fixed code blocking all injection attempts
Screenshot 5	Package upgraded to 1.83.10 (patched version)
Screenshot 6	Side-by-side before vs after proof

Quick Reference: What to Write in Your Report

CVE: CVE-2026-42208 **Severity:** CVSS 9.3 Critical **Affected:** LiteLLM Proxy v1.81.16 -- v1.83.6 **Patched in:** v1.83.7 (recommend v1.83.10-stable) **Weakness:** CWE-89 SQL Injection **Root cause:** The Authorization: Bearer header value was interpolated directly into an SQL query string (`f"SELECT ... WHERE token = '{bearer_token}'"`) instead of being passed as a separate parameter **Fix:** Replace string interpolation with a parameterized query (`cursor.execute("SELECT ... WHERE token = ?", (bearer_token,))`) so user input is never interpreted as SQL syntax **Impact:** Unauthenticated attacker can read and modify the LiteLLM proxy database, exposing all managed AI provider credentials (OpenAI keys, Anthropic keys, etc.)